

Database design

Database design is the organization of data according to a database model. The designer determines what data must be stored and how the data elements interrelate. With this information, they can begin to fit the data to the database model.^[1] A database management system manages the data accordingly.

Database design is a process that consists of several steps.

Conceptual data modeling

The first step of database design involves classifying data and identifying interrelationships. The theoretical representation of data is called an *ontology* or a *conceptual data model*.

Determining data to be stored

In a majority of cases, the person designing a database is a person with expertise in database design, rather than expertise in the domain from which the data to be stored is drawn e.g. financial information, biological information etc. Therefore, the data to be stored in a particular database must be determined in cooperation with a person who does have expertise in that domain, and who is aware of the meaning of the data to be stored within the system.

This process is one which is generally considered part of requirements analysis, and requires skill on the part of the database designer to elicit the needed information from those with the domain knowledge. This is because those with the necessary domain knowledge often cannot clearly express the system requirements for the database as they are unaccustomed to thinking in terms of the discrete data elements which must be stored. Data to be stored can be determined by Requirement Specification.^[2]

Determining data relationships

Once a database designer is aware of the data which is to be stored within the database, they must then determine where dependency is within the data. Sometimes when data is changed you can be changing other data that is not visible. For example, in a list of names and addresses, assuming a situation where multiple people can have the same address, but one person cannot have more than one address, the address is dependent upon the name. When provided a name and the list the address can be uniquely determined; however, the inverse does not hold – when given an address and the list, a name cannot be uniquely determined because multiple people can reside at an address. Because an address is determined by a name, an address is considered dependent on a name.

(NOTE: A common misconception is that the relational model is so called because of the stating of relationships between data elements therein. This is not true. The relational model is so named because it is based upon the mathematical structures known as relations.)

Conceptual schema

The information obtained can be formalized in a diagram or schema. At this stage, it is a conceptual schema.

ER diagram (entity–relationship model)

One of the most common types of conceptual schemas is the ER (entity–relationship model) diagrams.

Attributes in ER diagrams are usually modeled as an oval with the name of the attribute, linked to the entity or relationship that contains the attribute.

ER models are commonly used in information system design; for example, they are used to describe information requirements and / or the types of information to be stored in the database during the conceptual structure design phase.^[3]



A sample entity–relationship diagram

Logical data modeling

Once the relationships and dependencies amongst the various pieces of information have been determined, it is possible to arrange the data into a logical structure which can then be mapped into the storage objects supported by the database management system. In the case of relational databases the storage objects are tables which store data in rows and columns. In an Object database the storage objects correspond directly to the objects used by the Object-oriented programming language used to write the applications that will manage and access the data. The relationships may be defined as attributes of the object classes involved or as methods that operate on the object classes.

The way this mapping is generally performed is such that each set of related data which depends upon a single object, whether real or abstract, is placed in a table. Relationships between these dependent objects are then stored as links between the various objects.

Each table may represent an implementation of either a logical object or a relationship joining one or more instances of one or more logical objects. Relationships between tables may then be stored as links connecting child tables with parents. Since complex logical relationships are themselves tables they will probably have links to more than one parent.

Normalization

In the field of relational database design, *normalization* is a systematic way of ensuring that a database structure is suitable for general-purpose querying and free of certain undesirable characteristics—insertion, update, and deletion anomalies that could lead to loss of data integrity.

A standard piece of database design guidance is that the designer should create a fully normalized design; selective denormalization can subsequently be performed, but only for performance reasons.^[4] The trade-off is storage space vs performance. The more normalized the design is, the less data redundancy there is (and therefore, it takes up less space to store), however, common data retrieval patterns may now need complex joins, merges, and sorts to occur – which takes up more data read, and compute cycles. Some modeling disciplines, such as the dimensional modeling approach to data warehouse design, explicitly recommend non-normalized designs, i.e. designs that in large part do not adhere to 3NF. Normalization consists of normal forms that are 1NF, 2NF, 3NF, Boyce-Codd NF (3.5NF), 4NF, 5NF and 6NF.

Document databases take a different approach. A document that is stored in such a database, typically would contain more than one normalized data unit and often the relationships between the units as well. If all the data units and the relationships in question are often retrieved together, then this approach optimizes the number of retrieves. It also simplifies how data gets replicated, because now there is a clearly identifiable unit of data whose consistency is self-contained. Another consideration is that reading and writing a single document in such databases will require a single transaction – which can be an important consideration in a Microservices architecture. In such situations, often, portions of the document are retrieved from other services via an API and stored locally for efficiency reasons. If the data units were to be split out across the services, then a read (or write) to support a service consumer might require more than one service calls, and this could result in management of multiple transactions, which may not be preferred.

Physical design

Physical data modeling

The physical design of the database specifies the physical configuration of the database on the storage media. This includes detailed specification of data elements and data types.

Other physical design

This step involves specifying the indexing options and other parameters residing in the DBMS data dictionary. It is the detailed design of a system that includes modules & the database's hardware & software specifications of the system. Some aspects that are addressed at the physical layer:

- Performance – mainly addressed via indexing for the read/update/delete queries, data type choice for insert queries
- Replication – what pieces of data get copied over into another database, and how often. Are there multiple-masters, or a single one?
- High-availability – whether the configuration is active-passive, or active-active, the topology, coordination scheme, reliability targets, etc all have to be defined.
- Partitioning – if the database is distributed, then for a single entity, how is the data distributed amongst all the partitions of the database, and how is partition failure taken into account.
- Backup and restore schemes.

At the application level, other aspects of the physical design can include the need to define stored procedures, or materialized query views, OLAP cubes, etc.

See also

- [Concept mapping](#)
- [Data modeling](#)
- [Database normalization](#)
- [Entity–relationship model](#)
- [Entity-attribute-value model](#)
- [Knowledge representation](#)
- [Logical data model](#)
- [Mindmap](#)
- [Object–relationship modeling](#)
- [Object–role modeling](#)
- [Physical data model](#)
- [Principle of orthogonal design \(POOD\)](#)
- [Relational database](#)
- [Relational model](#)
- [Semantic Web](#)
- [Three-schema approach](#)

References

1. Teorey, T.J., Lightstone, S.S., et al., (2009). Database Design: Know it all.1st ed. Burlington, MA.: Morgan Kaufmann Publishers
2. Teorey, T.; Lightstone, S. and Nadeau, T.(2005) *Database Modeling & Design: Logical Design*, 4th edition, Morgan Kaufmann Press. ISBN 0-12-685352-5
3. Javed, Muhammad; Lin, Yuqing (2018). "Iterative Process for Generating ER Diagram from Unrestricted Requirements". *Proceedings of the 13th International Conference on Evaluation of Novel Approaches to Software Engineering*. SCITEPRESS – Science and Technology Publications. pp. 192–204. doi:10.5220/0006778701920204 (<https://doi.org/10.5220%2F0006778701920204>). ISBN 978-989-758-300-1.
4. Zhou, Fang; Gao, Ying (2024). "Performance Study on Normalization and Denormalization in MES System Databases". *2024 International Conference on Intelligent Computing and Data Mining (ICDM)*. pp. 90–94. Bibcode:2024icdm.conf...24Z (<https://ui.adsabs.harvard.edu/abs/2024icdm.conf...24Z>). doi:10.1109/ICDM63232.2024.10762243 (<https://doi.org/10.1109%2FICDM63232.2024.10762243>). ISBN 979-8-3315-0626-1.

Further reading

- S. Lightstone, T. Teorey, T. Nadeau, "Physical Database Design: the database professional's guide to exploiting indexes, views, storage, and more", Morgan Kaufmann Press, 2007. ISBN 0-12-369389-6
- M. Hernandez, "Database Design for Mere Mortals (<http://www.informit.com/store/database-design-for-mere-mortals-a-hands-on-guide-to-9780321884497>): A Hands-On Guide to Relational Database Design", 3rd Edition, Addison-Wesley Professional, 2013. ISBN 0-321-88449-3

External links

- [\[1\]](#)
- [\[2\]](#)
- Database Normalization Basics (<http://databases.about.com/od/specificproducts/a/normalization.htm>) Archived (<https://web.archive.org/web/20070205184226/http://databases.about.com/od/specificproducts/a/normalization.htm>) 2007-02-05 at the [Wayback Machine](#) by Mike Chapple (About.com)

- Database Normalization Intro (<http://www.databasejournal.com/sql/etc/article.php/1428511>) Archived (<https://web.archive.org/web/20110928000244/http://www.databasejournal.com/sql/etc/article.php/1428511>) 2011-09-28 at the Wayback Machine, Part 2 (http://www.databasejournal.com/sql/etc/article.php/26861_1474411_1) Archived (https://web.archive.org/web/20110708233620/http://www.databasejournal.com/sql/etc/article.php/26861_1474411_1) 2011-07-08 at the Wayback Machine
- "An Introduction to Database Normalization" (<https://web.archive.org/web/20110606025027/http://dev.mysql.com/tech-resources/articles/intro-to-normalization.html>). Archived from the original (<http://dev.mysql.com/tech-resources/articles/intro-to-normalization.html>) on 2011-06-06. Retrieved 2012-02-25.
- "Normalization" (<https://web.archive.org/web/20100106115112/http://www.utexas.edu/its/archive/windows/database/datamodeling/rm/rm7.html>). Archived from the original (<http://www.utexas.edu/its/windows/database/datamodeling/rm/rm7.html>) on 2010-01-06. Retrieved 2012-02-25.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Database_design&oldid=1363262630"